Personal Finance Analyzer

Course Project - IT Workshop III

1. Problem Statement and Objectives

A bank statement lists every transaction but does not show **where the money goes**. The aim of this project is to build a Python tool that takes a bank statement CSV and finds out where the money goes. The tool should:

- clean the raw statement and put every transaction into a spending category;
- show spending by category and by month, and calculate savings;
- find unusual (anomaly) expenses and repeating (recurring) payments;
- check spending against a monthly budget and forecast next month's spending;
- store the cleaned data in a SQLite database and answer questions with SQL.

Tools: Python, Jupyter Notebook, pandas, NumPy, Matplotlib, scikit-learn (Linear Regression), sqlite3.

2. Dataset

The notebook was run on a bank statement file (`bank_statement.csv`) with **437 transactions** from **Jan-Dec 2025**. Columns: `date, description, amount, type` (debit/credit). Two settings are kept in one cell so they are easy to change: `CSV_FILE` (the file name; a Kaggle statement can be placed here) and `DAY_FIRST` (True for dates like 25-12-2025, False for 2025-12-25). A separate cell creates a small *synthetic* statement, but **only if the file does not exist**; in this run the existing file was used.

3. Methodology

The project is a pipeline of eight steps on one pandas DataFrame:

Clean -> Categorise -> Charts and savings -> Anomaly detection -> Recurring payments -> Budget check -> Forecast -> SQLite

- **Cleaning:** remove duplicate rows and rows with empty cells, convert *date* to a real date, strip and upper-case descriptions, make amounts positive, standardise *type* to debit/credit (dr/cr/withdrawal/deposit are renamed), sort by date and add a *month* column such as 2025-03.
- **Categorisation:** a dictionary maps keywords to 8 categories (Food, Travel, Subscriptions, Shopping, Rent, Health, Entertainment, Bills). The **first match wins**, so the order matters (PRIME is checked in Subscriptions before AMAZON in Shopping). No match gives Others.
- **Charts and savings:** only debit rows count as spending. Savings rate = (money in - money out) / money in x 100.
- **Anomaly rule:** within each category separately, a debit is unusual if it is above *mean + 2 x standard deviation* (categories with under 5 rows are skipped).
- **Recurring rule:** digit-only words are removed from the description to get a merchant name; a merchant seen in 3 or more months whose standard deviation is below 5% of its mean amount is recurring. Yearly cost = mean amount x 12.
- **Budget rule:** one month is compared with a fixed limit per category: below 80% = OK, 80% or more = WARNING, 100% or more = ALERT.
- **Forecast:** Linear Regression with x = month number (1-12) and y = monthly spending; predict month 13.
- **Storage:** the cleaned table is saved to `finance.db` (dates stored as text) and queried with SQL.

4. Implementation (key code)

```
df = df.drop_duplicates()
df = df.dropna(subset=["date", "description", "amount", "type"])
df["date"] = pd.to_datetime(df["date"], dayfirst=DAY_FIRST)         # cleaning
df["amount"] = df["amount"].abs()
df["month"] = df["date"].dt.strftime("%Y-%m")

def find_category(description):                                     # categorisation
    for category in category_keywords:
        for word in category_keywords[category]:
            if word in description:
                return category
    return "Others"

limit = group["amount"].mean() + 2 * group["amount"].std()         # anomaly threshold
big = group[group["amount"] > limit]

if months_seen >= 3 and std < 0.05 * mean:                          # recurring rule
    recurring_list.append({"merchant": merchant, "yearly_cost": round(mean * 12, 2)})

percent = 100 * spent / limit                                       # budget status
status = "ALERT" if percent >= 100 else "WARNING" if percent >= 80 else "OK"

model = LinearRegression().fit(x, y)                                # forecast
prediction = model.predict([[next_month_number]])[0]

table_to_save.to_sql("transactions", connection, if_exists="replace", index=False)   # SQLite
```

5. Results

5.1 Cleaning and categories. 437 rows were loaded and 437 remained after cleaning (no duplicates or blank cells). The only types found were debit and credit. Category counts (all rows): Travel 88, Food 88, Bills 80, Others 45, Subscriptions 36, Health 33, Shopping 33, Entertainment 22, Rent 12. The 45 rows in Others include non-spending rows such as SALARY CREDIT. Spending (debit) by category:

Category	Debit total (Rs.)	Share
Rent	1,37,995.32	28.9%
Shopping	71,841.06	15.0%
Others	58,396.25	12.2%
Bills	51,408.97	10.8%
Health	50,152.82	10.5%
Food	45,783.29	9.6%
Travel	35,347.57	7.4%
Entertainment	14,232.86	3.0%
Subscriptions	12,807.97	2.7%
Total spending	**4,77,966.11**	**100%**

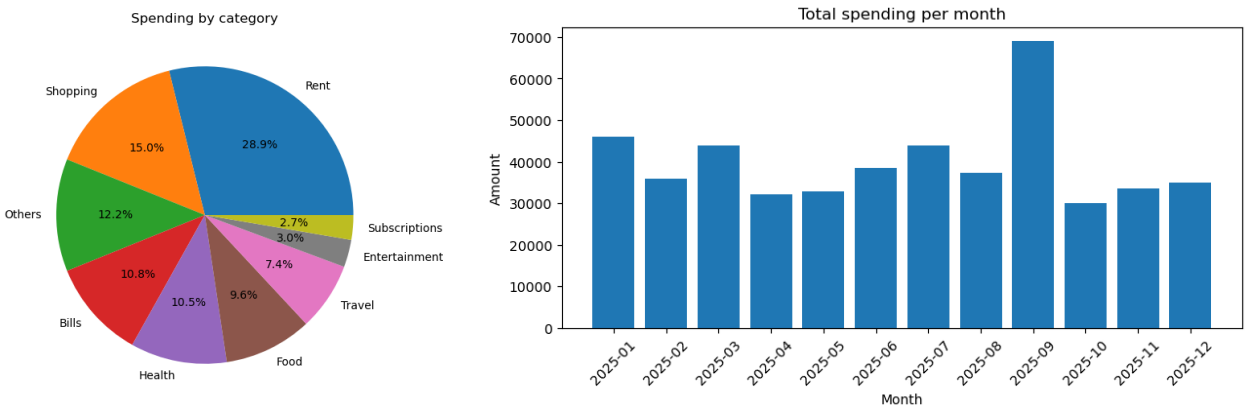


Figure 1: Spending by category (left) and total spending per month (right).

5.2 Savings and top expenses. Money in Rs. 6,52,775, money out Rs. 4,77,966, savings Rs. 1,74,809, **savings rate 26.8%**. Average monthly spending is about Rs. 39,830. **September (about Rs. 69,000) is the highest month** because of one Rs. 28,000 purchase at Reliance Digital; October is the lowest (about Rs. 30,000). The five biggest single expenses are:

Date	Description	Category	Amount (Rs.)
2025-09-07	RELIANCE DIGITAL	Others	28,000.00
2025-03-18	FLIPKART ORDER	Shopping	15,000.00
2025-12-04	HOUSE RENT PAID	Rent	11,598.69
2025-11-06	HOUSE RENT PAID	Rent	11,578.58
2025-09-05	HOUSE RENT PAID	Rent	11,567.18

Only the first two are one-off purchases; the other three are the regular monthly rent.

5.3 Unusual expenses (10 flagged). The rule flagged ten expenses:

Date	Description	Category	Amount (Rs.)	Verdict
2025-09-07	RELIANCE DIGITAL	Others	28,000.00	Genuine
2025-03-18	FLIPKART ORDER	Shopping	15,000.00	Genuine
2025-06-22	RESTAURANT FAMILY BILL	Food	9,000.00	Genuine
2025-07-14	APOLLO PHARMACY	Health	6,800.00	Genuine
2025-11-08	IRCTC TICKET	Travel	4,500.00	Genuine
2025-09-09	KSEB BILL	Bills	2,193.00	False positive
2025-05-10	KSEB BILL	Bills	1,893.00	False positive
2025-12-15	KSEB BILL	Bills	1,890.00	False positive
2025-02-10	KSEB BILL	Bills	1,665.00	False positive
2025-03-08	PVR CINEMAS	Entertainment	1,176.07	Minor

The first five are real one-off expenses and make sense. The four KSEB electricity bills are **false positives**: the Bills category mixes small payments (broadband, recharge) with larger electricity bills, which pulls the threshold too low. The PVR bill is a minor outlier for the same reason in a small category.

5.4 Recurring payments. Five payments repeat every month, together **Rs. 1,80,822 per year (37.8% of spending)**. KSEB is correctly not listed because its amount changes every month.

Merchant	Months	Avg (Rs.)	Yearly cost (Rs.)
HOUSE RENT PAID	12	11,499.61	1,37,995.32
FITNESS GYM MEMBERSHIP	12	2,501.55	30,018.57
NETFLIX SUBSCRIPTION	12	648.94	7,787.22
AMAZON PRIME MEMBERSHIP	12	299.67	3,596.07
SPOTIFY PREMIUM	12	118.72	1,424.68
Total			**1,80,821.86**

5.5 Budget check (March 2025). Seven of nine categories were OK. Two need attention:

Category	Limit (Rs.)	Spent (Rs.)	% used	Status
Food	8,000	3,334.80	41.7	OK
Travel	5,000	3,722.60	74.5	OK
Bills	6,000	3,454.76	57.6	OK
Shopping	10,000	15,000.00	150.0	ALERT
Subscriptions	1,500	1,062.25	70.8	OK
Rent	12,000	11,434.52	95.3	WARNING
Health	4,000	2,505.19	62.6	OK
Entertainment	3,000	1,176.07	39.2	OK
Others	3,000	2,126.65	70.9	OK

The Shopping alert is caused by the single Rs. 15,000 Flipkart order; Rent is a fixed amount close to its limit.

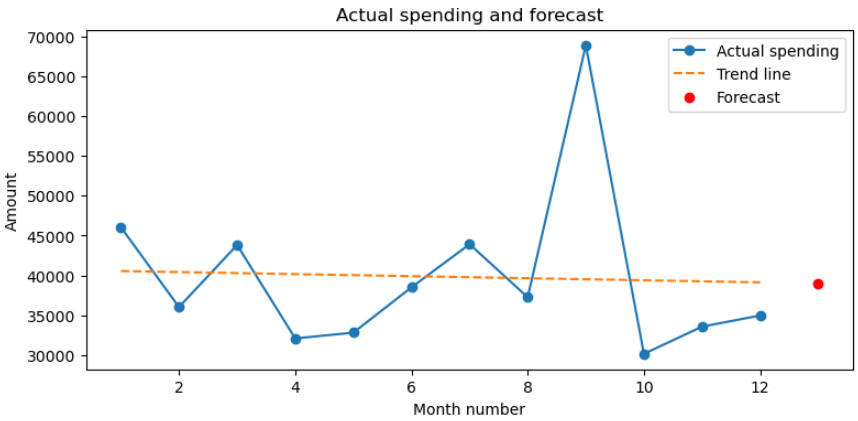


Figure 2: Monthly spending, trend line and forecast (month 13).

5.6 Forecast. The trend line is almost flat and slightly downward, so spending is neither rising nor falling much. Predicted spending for next month: **Rs. 38,990.85**, close to the yearly average. The forecast is believable as a rough guide, but with only 12 points it is not a promise.

5.7 SQL queries. The cleaned table (437 rows, with the added month, category, is_anomaly and merchant columns) was saved to `finance.db`. Two queries were run:

```
-- Query 1: total spending per category
SELECT category, ROUND(SUM(amount), 2) AS total_spent FROM transactions
WHERE type = 'debit' GROUP BY category ORDER BY total_spent DESC

-- Query 2: top 5 merchants by total spending
SELECT merchant, COUNT(*) AS times, ROUND(SUM(amount), 2) AS total_spent FROM transactions
WHERE type = 'debit' GROUP BY merchant ORDER BY total_spent DESC LIMIT 5
```

Query 1 matched the pandas category totals in section 5.1 exactly. Query 2 result:

Merchant	Times	Total (Rs.)
HOUSE RENT PAID	12	1,37,995.32
AMAZON ORDER	17	38,503.54
FLIPKART ORDER	10	33,337.52
FITNESS GYM MEMBERSHIP	12	30,018.57
RELIANCE DIGITAL	1	28,000.00

6. Conclusion

The tool cleaned a raw bank statement, put each transaction into a category, found unusual and repeating payments, checked a budget, forecast next month's spending and saved everything in SQLite. Key findings: **rent and recurring payments (37.8%) dominate spending**, shopping is the largest flexible category, **one-time purchases (Reliance Digital, Flipkart) cause the monthly spikes and budget alerts**, the savings rate is a healthy **26.8%**, and next month is forecast at about **Rs. 38,991**.

Problems faced

- **Date format:** statements use either 25-12-2025 or 2025-12-25, which pandas can read wrongly. This was solved with the *DAY_FIRST* setting.
- **Order of categories:** AMAZON PRIME MEMBERSHIP contains both PRIME and AMAZON. Because the first match wins, Subscriptions is checked before Shopping, so it is not counted as shopping.
- **Merchant names:** descriptions carry random numbers (SWIGGY ORDER 4821), so digit-only words were removed before grouping by merchant.
- **False positives:** the mean + 2 x std rule flagged normal electricity bills (section 5.3).

Limitations

- Keyword rules do not know new merchants; they fall into Others until a keyword is added.
- The forecast uses only 12 points, so it is only a rough trend.
- Mean + 2 x std works best when amounts are roughly bell shaped; outliers distort it and cause false positives.
- Budget limits are fixed by hand and only one month is checked at a time.
- Synthetic sample data is cleaner than a real bank statement.

Future scope: a web dashboard (Gradio), learning categories with machine learning, reading PDF statements, and per-merchant anomaly detection.